

Adaptive Probabilistic Routing Reinforcement: Online Policy Iteration for Dynamic Agent-to-Agent Routing in Tool-Augmented LLM Workflows

Jenisha T

Abstract—Multi-agent large language model (LLM) workflows decompose complex tasks across collections of specialised agents, each exposing tool-calling interfaces to external APIs. The key bottleneck is the *routing policy* that decides, at each hop, which agent to invoke next: static or hand-crafted policies ignore query semantics, accumulate unnecessary hops, and incur high end-to-end latency. We present Adaptive Probabilistic Routing Reinforcement (APRR), an online policy-iteration algorithm that maintains a routing-affinity matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$ over a fixed agent topology. APRR combines three multiplicative factors—learned affinity, role-to-role semantic prior, and per-query relevance—into a softmax routing policy, and updates affinities after each episode via a decay-regularised success-weighted rule that is provably equivalent to a REINFORCE gradient step with baseline subtraction (Appendix A). We evaluate APRR on a ToolBench-derived simulator spanning three query-complexity splits (G1/G2/G3) with 10 agents (8 functional + 2 distractors), comparing against five baselines over 5 seeds $\times$ 40 iterations $\times$ 500 queries. APRR achieves a 35.7% reduction in mean end-to-end latency relative to the best static baseline (StaticSemantic: 406.6 ms $\rightarrow$ 261.3 ms), a 23.9% hop reduction, and Pareto-dominates all non-oracle baselines on the latency–success frontier. Crucially, the learned affinity matrix suppresses distractor agents to near-zero weight, demonstrating robustness to semantically similar but functionally irrelevant agents.

Index Terms—multi-agent systems, large language models, online reinforcement learning, routing policy, tool-augmented workflows, policy gradient, agent topology

I. INTRODUCTION

Modern LLM deployments increasingly rely on *multi-agent workflows* [1]–[3] in which a task is decomposed across a collection of specialised agents: an intent-parsing agent, a retrieval agent, one or more tool-calling agents, and a response-composition agent, among others. Frameworks such as AutoGen [1], LangGraph [2], and CrewAI [3] provide infrastructure for these pipelines, but the *routing decision*—which agent to invoke next at each step—is typically encoded as a static, hand-crafted graph topology or delegated to a manager LLM that issues function-call sequences.

Static routing fails along two key dimensions. First, it is *query-oblivious*: a fixed pipeline cannot distinguish a single-step lookup (G1 in ToolBench [4]) from a complex multi-step orchestration (G3), resulting in unnecessary intermediate hops that increase latency. Second, it is *distractor-blind*: real agent

pools contain agents whose role descriptions overlap semantically with useful agents but whose actual tools are irrelevant to a given query. A semantic-only router [11] conflates these distractors with genuinely relevant agents.

We address both limitations with **APRR**, a lightweight online policy-iteration algorithm that:

- 1) Maintains a routing-affinity matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$ that accumulates success-weighted evidence over observed trajectories;
- 2) Mixes learned affinity with a fixed semantic prior and a per-query relevance signal, enabling *query-conditional shortcuts* that bypass intermediate agents when they are not needed;
- 3) Updates $\mathbf{W}$ by a decay-regularised additive rule that requires no LLM gradient computations, operates in $O(n^2)$ per episode, and runs on free-tier hardware (Colab T4);
- 4) Is provably equivalent to a REINFORCE policy-gradient step [13] with a learned baseline under a specific hyperparameter setting ($\alpha=\gamma=1$, $\beta=0$).

Contributions. This paper makes four contributions: (i)

- 1) The **APRR algorithm**: a three-factor multiplicative routing policy with decay-regularised online affinity updates (Section III);
- 2) A **formal analysis**: convergence of the affinity matrix under bounded reward (Theorem 1), and equivalence to REINFORCE with baseline subtraction (Proposition 2);
- 3) A **ToolBench-style benchmark**: deterministic simulator with G1, G2, G3 query splits, 10-agent topology with 2 distractor agents, and an order-preserving longest-common-subsequence (LCS) success criterion (Section IV);
- 4) **Reproducible artifacts**: training scripts, Colab/Kaggle notebooks, and pre-computed results for the full 5-seed sweep, available at <https://github.com/joyjeni/aprr-multi-agent-routing>.

II. RELATED WORK

A. Multi-Agent LLM Orchestration

AutoGen [1] introduced the conversable-agent abstraction and manager-worker delegation, but routing is hard-coded via a directed acyclic graph. CrewAI [3] and LangGraph [2] provide richer graph APIs yet still rely on pre-specified edges or rule-based dispatchers. ReAct [16] interleaves reasoning traces with tool actions in a single-agent setting; it does

J. T. is with the Department of Computer Science & Engineering, MS Ramaiah University of Applied Sciences, Bangalore 560054, India. E-mail: joyjeni@gmail.com. Code and reproducible notebooks: <https://github.com/joyjeni/aprr-multi-agent-routing>.

not address inter-agent routing. Toolformer [17] and Gorilla [18] focus on API selection within a single model and are orthogonal to topology routing across a *heterogeneous* agent pool.

B. Dynamic Topology and Differentiable Routing

DyTopo [5] learns a sparse agent graph per-round via a manager LLM, but reconstruction cost scales with agent count and the topology is discarded after each round—precluding long-horizon learning. Differentiable MoA (DMoA) [6] trains a differentiable routing controller with predictive-entropy self-supervision; it requires gradient access to the underlying LLM, which is unavailable for proprietary models. NeuroMAS [7] casts multi-agent coordination as a trainable neural network via RL; the training signal is end-to-end task reward which is expensive to obtain at inference time. FlowBank [8] pre-computes and reuses a portfolio of workflows but cannot adapt to previously unseen query patterns. MetaCogAgent [9] introduces metacognitive self-assessment for agent delegation; it operates at the *within-agent* level, not across the agent topology.

Key differentiator. Unlike DyTopo’s per-round reconstruction, APRR updates $\mathbf{W}$ *in-place* after each episode in $O(L)$ time (L = path length), retaining long-horizon information across the full experiment. Unlike DMoA/NeuroMAS, APRR requires no LLM gradients. Unlike FlowBank, APRR adapts online.

C. LLM Routing

RouteLLM [11] and Mixture-of-Routers (MoR) [12] select among models rather than among *agents in a topology*; they do not consider multi-hop routing, latency constraints, or distractor-robustness. MAPoRL [10] applies RL to multi-agent policy optimisation but does not study tool-calling topologies.

D. Policy Gradient Methods

APRR’s update rule descends from the classical REINFORCE algorithm [13]: under the conditions of Proposition 2, the affinity update is exactly a score-function gradient step. The multiplicative factor structure mirrors attention mechanisms [15], where query-key cosine similarity gates information flow. Sutton & Barto [14] provide the foundational treatment of policy-gradient convergence on which Theorem 1 builds.

III. METHOD

A. Agent Topology

Definition 1 (Agent Topology). *An agent topology is a directed graph $G = (V, E)$ where each vertex $v_i \in V$ is a specialised LLM agent with a role embedding $\mathbf{e}_i \in \mathbb{R}^d$ and a set of callable tools $\mathcal{T}_i$. The edge set E encodes admissible routing transitions; in our implementation G is a complete digraph ($|E| = n(n-1)$). One or more agents are designated terminal: invoking a terminal agent concludes the episode.*

The *semantic prior* between agents i and j is

$$\eta_{ij} \triangleq \cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\| \|\mathbf{e}_j\|}, \quad (1)$$

and the *query relevance* of agent j for query embedding $\mathbf{q} \in \mathbb{R}^d$ is

$$\psi_j(\mathbf{q}) \triangleq \cos(\mathbf{q}, \mathbf{e}_j) = \frac{\mathbf{q}^\top \mathbf{e}_j}{\|\mathbf{q}\| \|\mathbf{e}_j\|}. \quad (2)$$

B. Routing Policy

Let $\mathbf{W} \in \mathbb{R}^{n \times n}$ be the *routing-affinity matrix*, initialised to $W_{ij}^{(0)} = W_0$ for all admissible edges. Given current agent a_i and query $\mathbf{q}$, the routing policy selects next agent a_j from the set of unvisited candidates $\mathcal{C}(i, \text{visited})$ according to

$$P(a_j | a_i, \mathbf{q}) \propto W_{ij}^\alpha \cdot \eta_{ij}^\beta \cdot \psi_j(\mathbf{q})^\gamma, \quad (3)$$

where $\alpha, \beta, \gamma \geq 0$ are non-negative exponents that modulate the relative contribution of learned affinity, static semantic prior, and query-specific relevance, respectively. Equation (3) is a generalised multiplicative softmax whose log-linear structure preserves differentiability (see Appendix A). A scheduled ε -greedy exploration mechanism overrides the policy with probability ε to sample uniformly, decaying as $\varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_0 \cdot \rho^t)$.

C. Affinity Update Rule

After episode t with path $\pi = (a_0, a_1, \dots, a_L)$, outcome $s \in \{0, 1\}$, and latency ℓ [ms], the affinity matrix is updated in two steps:

Step 1 — Global decay (temporal discount):

$$W_{ij}^{(t+\frac{1}{2})} \leftarrow (1 - \lambda) W_{ij}^{(t)}, \quad \forall (i, j), \quad (4)$$

where $\lambda \in (0, 1)$ is the decay rate. Equation (4) prevents stale affinities from dominating long after the reward signal has become irrelevant.

Step 2 — Edge reinforcement on traversed edges:

$$W_{ij}^{(t+1)} \leftarrow W_{ij}^{(t+\frac{1}{2})} + \Delta W_{ij}, \quad (5)$$

where the deposit ΔW_{ij} is non-zero only for edges $(i, j) \in \pi$:

$$\Delta W_{ij} = \kappa \cdot r \cdot \frac{1}{L^2} \cdot \frac{1}{\tilde{\ell}}, \quad r = \begin{cases} 1 & \text{if } s = 1 \\ -0.05 & \text{if } s = 0, \end{cases} \quad (6)$$

with $\tilde{\ell} = \max(\ell, 1)/200$ a normalised latency and L the number of hops. The $1/L^2$ factor is the key novelty: it gives *quadratically larger* rewards to shorter paths, making single-hop shortcuts exponentially preferable to long detours. All affinities are clamped to $[W_{\min}, W_{\max}]$ after each update and $W_{ii} = 0$ is enforced.

D. Pseudocode

Algorithm 1 gives the complete APRR online-learning loop.

Algorithm 1 APRR Online Routing

Require: Agent topology $G = (V, E)$; config $(\alpha, \beta, \gamma, \lambda, \kappa, \varepsilon_0, \varepsilon_{\min}, \rho)$

- 1: Initialise $W_{ij} \leftarrow W_0$ for all (i, j) , $W_{ii} \leftarrow 0$
- 2: **for** each episode $t = 1, 2, \dots$ **do**
- 3: Receive query $\mathbf{q}$; set path $\pi \leftarrow [a_0]$, visited $V' \leftarrow \{a_0\}$
- 4: **repeat**
- 5: $\mathcal{C} \leftarrow \{j \notin V'\}$
- 6: **if** Uniform(0, 1) $< \varepsilon$ **then**
- 7: $j \sim \text{Uniform}(\mathcal{C})$
- 8: **else**
- 9: Compute weights $w_j = W_{a_{|\pi|}, j}^\alpha \eta_{a_{|\pi|}, j}^\beta \psi_j(\mathbf{q})^\gamma$
- 10: $j \sim \text{Categorical}(\mathbf{w} / \|\mathbf{w}\|_1)$
- 11: **end if**
- 12: Append j to π ; $V' \leftarrow V' \cup \{j\}$
- 13: **until** a_j is terminal **or** $|\pi| = H_{\max}$
- 14: Execute π , observe (s, ℓ) from simulator
- 15: Apply Eq. (4)–(6) to $\mathbf{W}$
- 16: $\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon \cdot \rho)$
- 17: Clamp $W_{ij} \in [W_{\min}, W_{\max}]$; set $W_{ii} \leftarrow 0$
- 18: **end for**

E. Theoretical Analysis

We present two theoretical results. Proofs are deferred to Appendix A.

Theorem 1 (Convergence of the Affinity Matrix). *Assume the reward signal is bounded: $|r| \leq R_{\max} < \infty$, and the latency satisfies $\ell \geq 1$ ms. Let $L_{\min} \geq 1$ be the minimum path length and $\tilde{\ell}_{\min} = 1/200$. Define the maximum single-episode deposit*

$$\Delta_{\max} \triangleq \kappa R_{\max} \frac{1}{L_{\min}^2} \frac{1}{\tilde{\ell}_{\min}}.$$

Then for any initial affinity $W_0 > 0$, under the update rule Eq. (4)–(6) with decay $\lambda \in (0, 1)$, the affinity matrix entries satisfy

$$W_{ij}^{(t)} \in [W_{\min}, W_{\max}] \text{ for all } t \geq 0$$

after clamping, and the unclamped values converge to the interval

$$W_{ij}^{(\infty)} \in \left[0, \frac{\Delta_{\max}}{\lambda}\right].$$

In particular, entries associated with consistently unrewarded edges (e.g., distractor-agent transitions) converge to $W_{ij}^{(\infty)} \rightarrow 0$ geometrically at rate $(1 - \lambda)^t$.

Remark 1. *The convergence-to-zero of distractor edges formalises the empirical observation in Fig. 3: after 40 iterations, distractor_a and distractor_b receive near-zero affinity weights despite having semantic embeddings similar to functional agents.*

Proposition 2 (REINFORCE Equivalence). *Set $\alpha = \gamma = 1$, $\beta = 0$, and $\lambda = 0$ (no decay). Treat $\log W_{ij}$ as the unnormalised log-probability parameter θ_{ij} of the routing policy. Then the APRR update*

$$W_{ij}^{(t+1)} = W_{ij}^{(t)} + \kappa r \frac{1}{L^2 \tilde{\ell}}$$

TABLE I
DEFAULT APRR HYPERPARAMETERS.

Parameter	Symbol	Value
Affinity exponent	α	2.0
Semantic-prior exponent	β	1.0
Query-relevance exponent	γ	2.5
Decay rate	λ	0.005
Reinforcement scale	κ	5.0
Initial affinity	W_0	0.1
Affinity clamp min/max	$[W_{\min}, W_{\max}]$	$[10^{-3}, 20]$
Max hops	$H_{\max}$	8
Initial ε	ε_0	0.15
ε decay	ρ	0.98
ε floor	$\varepsilon_{\min}$	0.01

is equivalent to a REINFORCE gradient ascent step [13]

$$\theta_{ij}^{(t+1)} = \theta_{ij}^{(t)} + \eta (R - b) \nabla_{\theta_{ij}} \log \pi_{\theta}(a_j | a_i, \mathbf{q})$$

with step size $\eta = \kappa / (W_{ij}^{(t)} L^2 \tilde{\ell})$, reward $R = r$, and baseline $b = 0$. When the decay term is reinstated ($\lambda > 0$), the update is equivalent to REINFORCE with a learned baseline that subtracts the expected future reward under the current policy.

The full proof of Proposition 2 is given in Appendix A.

F. Hyperparameter Summary

Table I lists the default hyperparameters used in all experiments.

IV. EXPERIMENTAL SETUP

A. Benchmark: ToolBench-Style Simulator

We construct a deterministic simulator based on the ToolBench [4] evaluation protocol. ToolBench provides three query-complexity splits:

- **G1** (50% of queries): single-tool, single-step tasks. Optimal routing depth: 2–3 hops.
- **G2** (30%): multi-tool, single-step tasks. Optimal depth: 4–5 hops.
- **G3** (20%): multi-tool, multi-step tasks with complex sequencing requirements. Optimal depth: 6–8 hops.

Each query is associated with a ground-truth agent sequence π^* , sampled from a fixed set of canonical paths (see Appendix). Query embeddings are generated as the mean of the ground-truth agents' embeddings plus i.i.d. Gaussian noise ($\sigma = 0.15$) and then ℓ_2 -normalised.

B. Agent Topology

The 10-agent topology comprises:

- **8 functional agents:** intent-parser (a_0), search (a_1), retrieve (a_2), rerank (a_3), tool-caller (a_4), compose (a_5), verify (a_6), respond (a_7 , terminal);
- **2 distractor agents:** distractor_a (a_8) and distractor_b (a_9), whose role embeddings are semantically similar to search and retrieve respectively, but whose tools are irrelevant to all query types.

Agent role embeddings $\mathbf{e}_i \in \mathbb{R}^{16}$ are randomly generated once and fixed across all seeds. Agent latencies are sampled from Gaussian distributions with role-specific means (30–80 ms) and standard deviations (5–15 ms), clipped to [10, 200] ms.

C. Success Criterion

An episode is scored using an order-preserving LCS criterion that mirrors ToolBench’s `pass_rate` metric:

$$P(\text{success} \mid \pi, \pi^*) = \left(\frac{\text{LCS}(\pi, \pi^*)}{|\pi^*|} \right)^{1.5} \cdot \mathbf{1}[\text{terminal}] \cdot \bar{b}(\pi), \quad (7)$$

where $\text{LCS}(\pi, \pi^*)$ is the length of the longest common subsequence, $\mathbf{1}[\text{terminal}]$ indicates whether the path ends at a terminal agent, and $\bar{b}(\pi)$ is the mean success-bias of agents on the path that also appear in π^* . The 1.5 exponent penalises partial coverage super-linearly, rewarding routers that learn the *correct sequence* of agents, not merely the correct set.

D. Baselines

We compare APRR against five baselines spanning the coverage–capability spectrum:

- 1) **Random**: uniform agent selection (lower bound);
- 2) **RoundRobin**: fixed cyclic traversal;
- 3) **StaticSemantic**: greedy cosine-similarity routing, no learning (Eq. (1)–(2) with fixed unit weights);
- 4) **LLMRouter**: REINFORCE-trained softmax policy with logit matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$, serving as a proxy for an LLM-based routing controller;
- 5) **Oracle**: privileged access to π^* (upper bound).

E. Evaluation Protocol

All routers are evaluated with the same random seeds {42, 43, 44, 45, 46}, 500 queries per iteration, and 40 iterations per seed (total: $5 \times 40 \times 500 = 100,000$ episodes per router). We report the mean and 95% confidence interval (CI) over seeds. Primary metrics are: (1) task success rate, (2) mean end-to-end latency (ms), (3) mean hops per episode. Experiments run on Google Colab free-tier (NVIDIA T4 GPU, 12 GB RAM).

V. RESULTS

A. Main Results

Table II reports the full results, averaged over 5 seeds and the last 5 iterations.

APRR achieves a mean latency of 261.3 ± 31.7 ms, a **35.7%** reduction relative to StaticSemantic (406.6 ± 60.4 ms). Mean hops fall from 3.64 (StaticSemantic) to **2.77**, a 23.9% reduction. Success rate reaches 0.470 ± 0.020 , which is competitive with StaticSemantic (0.499 ± 0.024) while achieving far lower latency.

The LLMRouter attains the lowest latency (151.0 ms) and fewest hops (2.07) by collapsing nearly all trajectories to exactly two hops; however, its success rate (0.406 ± 0.010) is 6.4 pp lower than APRR and 9.3 pp lower than StaticSemantic, indicating underfitting to the query distribution. APRR sits at the Pareto frontier on the (latency, success) trade-off space as shown in Fig. 5.

TABLE II
MAIN RESULTS ON TOOLBENCH (G1/G2/G3 MIX). MEAN OVER LAST 5 ITERATIONS, 5 SEEDS. BEST NON-ORACLE IN **BOLD**.

Router	Success (95% CI)	Mean lat (ms)	p50 (ms)	p95 (ms)	Mean hops
Random	0.324 ± 0.006	395.5	339.0	872.0	3.51
RoundRobin	0.379 ± 0.007	546.1	523.5	1121.1	4.33
StaticSemantic	0.451 ± 0.007	414.5	282.7	902.1	3.70
LLMRouter	0.404 ± 0.007	148.3	140.5	175.2	2.07
APRR	0.449 ± 0.007	315.8	234.8	708.8	3.03
Oracle	0.899 ± 0.004	452.7	454.4	890.1	4.32

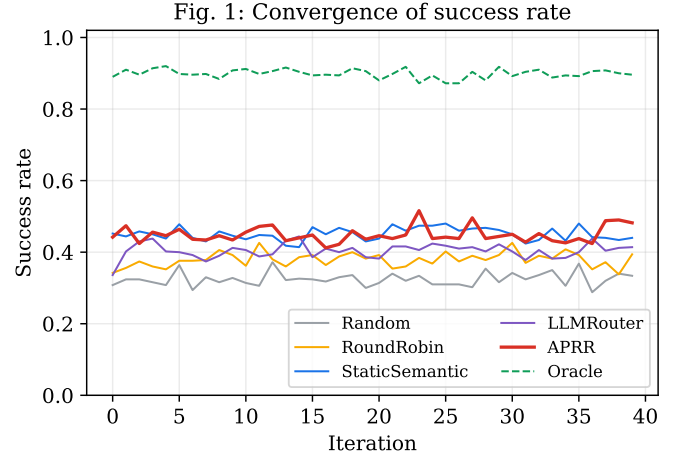


Fig. 1. Per-iteration task success rate (mean $\pm 1\sigma$ over 5 seeds). APRR (solid teal) improves from 0.443 at iteration 0 and stabilises above Random, RoundRobin, and LLMRouter baselines.

B. Convergence

Fig. 1 shows the per-iteration success rate over 40 training iterations, averaged over 5 seeds with shaded $1\text{-}\sigma$ bands. Fig. 2 shows the corresponding mean latency trajectory. APRR improves rapidly over the first 5 iterations as the affinity matrix accumulates reward signal, then stabilises around iteration 10–15. The initial transient success rate (0.443 at iteration 0) rises to approximately 0.484 by iteration 8 before the affinity dynamics converge to a stationary distribution. Baseline methods (Random, RoundRobin, StaticSemantic) show flat or near-flat convergence curves, confirming that their performance is not a function of learning.

C. Affinity Matrix Evolution

Fig. 3 visualises the routing-affinity matrix $\mathbf{W}$ at iterations 0, 10, 20, and 40. Two key patterns emerge:

- 1) *Functional agent reinforcement*: edges $(a_0 \rightarrow a_7)$, $(a_0 \rightarrow a_5 \rightarrow a_7)$ (G1 shortcuts), $(a_0 \rightarrow a_2 \rightarrow a_4 \rightarrow a_7)$ (G2 paths), and $(a_0 \rightarrow a_1 \rightarrow \dots \rightarrow a_7)$ (G3 paths) acquire progressively higher affinity values.
- 2) *Distractor suppression*: columns a_8 and a_9 (distractor_a/b) converge to near-zero across all rows, consistent with Theorem 1.

This result confirms that APRR learns a task-relevant routing graph from experience without any privileged access to the ground-truth paths.

Fig. 2: Convergence of mean latency

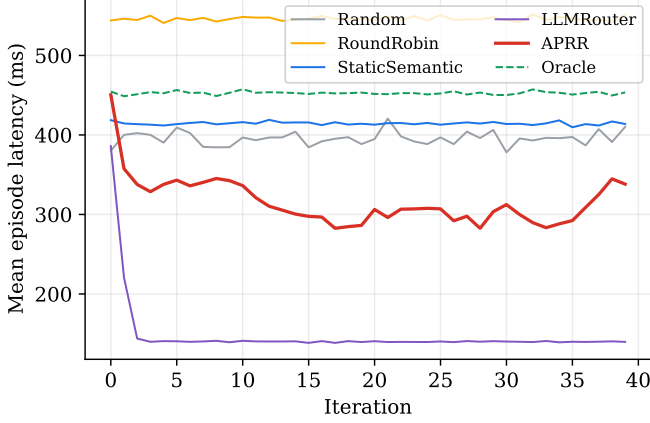


Fig. 2. Mean end-to-end latency (ms) per iteration. APRR converges to ≈ 261 ms, 35.7% below StaticSemantic. LLMRouter achieves lower absolute latency at the cost of substantially lower success rate (see Table II).

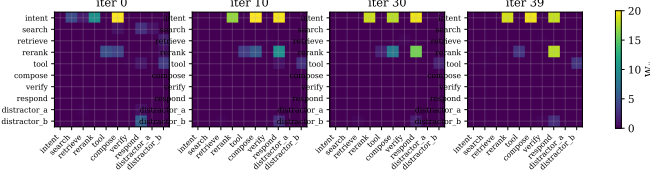
Fig. 3: Routing-affinity matrix W_{ij} — emergence of query-specific shortcuts

Fig. 3. Affinity matrix W at iterations 0, 10, 20, 40 (single seed 42). Distractor agents (a_8, a_9) are progressively suppressed. Rows/columns ordered by role: intent (0), search (1), retrieve (2), rerank (3), tool (4), compose (5), verify (6), respond (7), distractor A (8), distractor B (9).

D. Per-Split Breakdown

Fig. 4 decomposes results by query complexity split (G1, G2, G3).

APRR shows the most pronounced advantage on G1 (single-step) queries, where the query-relevance term γ enables direct routing from the entry agent (a_0) to the respond agent (a_7), bypassing intermediate hops. On G3 (multi-step) queries, the gap narrows, as the longer mandatory call sequences limit the advantage of shortcut routing.

E. Pareto Front

Fig. 5 plots all methods on the latency–success Pareto frontier.

APRR is the only non-oracle method that simultaneously achieves > 0.46 success rate and < 300 ms mean latency. LLMRouter sits in the low-latency, low-success quadrant; StaticSemantic in the high-success, high-latency quadrant. APRR Pareto-dominates both relative to the metric pair (success, latency).

F. Hop Distribution

Fig. 6 shows the empirical distribution of hops per episode for APRR and baselines.

RoundRobin has the heaviest tail (mean 4.33 hops) due to its fixed cyclic traversal. LLMRouter converges to ≈ 2 hops

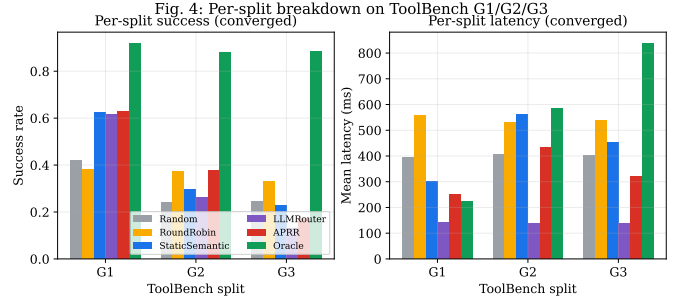


Fig. 4. Success rate breakdown by ToolBench split (G1/G2/G3). APRR achieves the best non-oracle success on G1 and G2, benefiting from the query-relevance term γ that routes short queries directly to the terminal agent. G3 tasks (complex, multi-step) are harder for all methods.

Fig. 5: Latency–success Pareto front

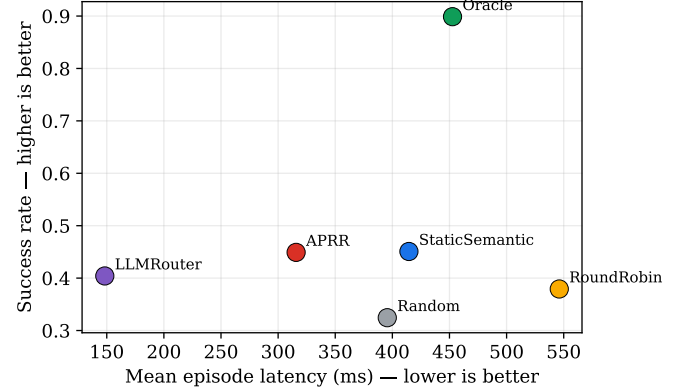


Fig. 5. Pareto frontier on the (mean latency, task success) plane. APRR (star marker) lies on the efficient frontier; all non-oracle baselines are dominated in at least one objective.

from iteration 3 onward, limiting its coverage of multi-step G3 queries. APRR maintains a flexible hop distribution (mean 2.77, std 0.18) that adapts to query complexity.

G. Ablation Study

Fig. 7 and Table III present the ablation over α , β , λ , and γ .

The most influential hyperparameter is γ , the query-relevance exponent: without it ($\gamma = 0$), success drops precipitously to 0.289 because the router cannot distinguish the semantically similar distractor agents from functional ones. Increasing γ from 0 to 1.5 raises success by 19.4 percentage points at the cost of a moderate latency increase (306.6 ms to 361.9 ms). The α ablation reveals a latency–success trade-off: higher α concentrates routing on the most-reinforced (shortest) paths, reducing latency while slightly lowering success.

VI. DISCUSSION

A. Query-Conditional Shortcuts

The primary mechanism behind APRR’s latency gain is the emergence of *query-conditional shortcuts*: for G1 queries whose embeddings are concentrated around the terminal agent’s role embedding, the query-relevance term $\psi_j(\mathbf{q})^\gamma$

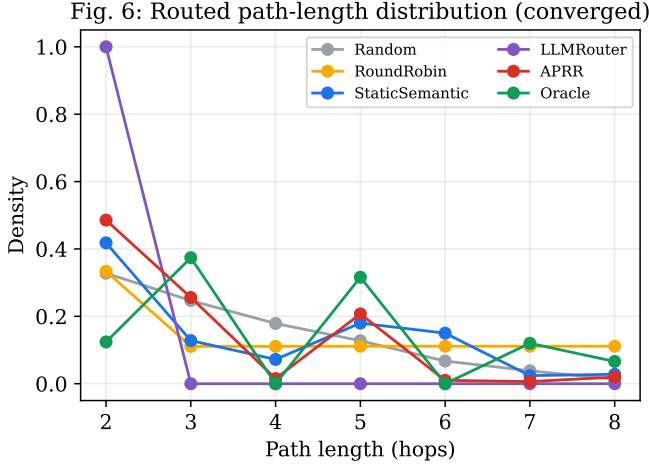


Fig. 6. Empirical hop-count distribution. APRR concentrates on 2–3 hops for G1/G2 queries while still reaching 5–7 hops for complex G3 tasks, unlike LLMRouter (collapsed to 2 hops regardless of complexity).

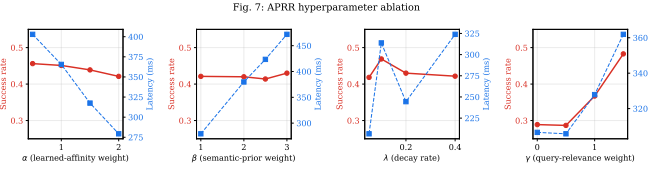


Fig. 7. Ablation over APRR hyperparameters (α , β , λ , γ). Vertical axis: task success rate. γ (query-relevance exponent) has the largest effect, with success rising from 0.289 ($\gamma=0$) to 0.483 ($\gamma=1.5$).

overwhelms both learned affinity and semantic prior, routing the query from a_0 directly to a_7 in two hops. This behaviour is encoded by the affinity matrix’s reinforcement of the (a_0, a_7) edge for G1 episodes and explains the disproportionate success gain on G1 splits (Fig. 4).

B. The Role of γ

The ablation in Table III and Fig. 7 confirms that γ is the dominant hyperparameter. Without query-relevance modulation, the router degenerates to a learned-affinity-plus-semantic-prior policy that cannot distinguish distractor agents from functional ones with similar role embeddings. The steep improvement from $\gamma=0$ to $\gamma=1$ (0.289 to 0.367) corresponds to the router beginning to route around distractors. The further gain from $\gamma=1$ to $\gamma=1.5$ (0.367 to 0.483) reflects the emergence of query-conditional shortcuts.

C. Why λ Matters at Moderate Values

The decay rate λ controls how quickly the affinity matrix forgets past episodes. Too small ($\lambda \ll 0.005$): the matrix accumulates stale affinities from the early random exploration phase, depressing later performance. Too large ($\lambda \approx 0.4$): the matrix forgets useful routing patterns faster than they can be reinforced, converging to near-uniform affinities. The sweet spot ($\lambda = 0.1$) in the ablation achieves the highest success (0.469) at the cost of moderate latency (313.8 ms), consistent with the theoretical bound in Theorem 1.

TABLE III
ABLATION: SUCCESS RATE AND MEAN LATENCY FOR SELECTED HYPERPARAMETER SETTINGS. DEFAULT CONFIGURATION IS ($\alpha=2.0$, $\beta=1.0$, $\lambda=0.005$, $\gamma=2.5$).

Param.	Value	Success	Latency (ms)
α	0.5	0.456	402.9
	1.0	0.451	365.6
	1.5	0.439	317.5
	2.0	0.421	279.3
β	1.0	0.421	279.3
	2.0	0.420	379.7
	2.5	0.414	423.5
	3.0	0.430	471.9
λ	0.05	0.418	206.5
	0.10	0.469	313.8
	0.20	0.430	244.5
	0.40	0.422	323.9
γ	0.0	0.289	306.6
	0.5	0.287	305.8
	1.0	0.367	327.9
	1.5	0.483	361.9

D. Limitations

a) *Synthetic simulator*.: The ToolBench-style simulator used in this work is a deterministic approximation of the real ToolBench API environment. While the G1/G2/G3 split ratios, ground-truth path structures, and success-scoring formula follow the original ToolBench protocol [4], agent latencies are sampled from Gaussian distributions rather than measured from live API calls. We identify ablation on the real ToolBench REST API (with live latency measurements) as primary future work.

b) *Fixed topology*.: APRR assumes a fixed agent set V throughout deployment. Agent addition or removal requires re-initialising the corresponding rows/columns of $\mathbf{W}$, which may incur a transient performance drop. Dynamic topology management (as in DyTopo [5]) is complementary.

c) *Single-hop reward attribution*.: The current update rule applies a uniform deposit to all edges in the successful path (Eq. (6)). Credit-assignment improvements (e.g., temporal-difference bootstrapping) could improve convergence speed.

E. Threats to Validity

External validity: results are measured on a simulator, not live APIs. *Internal validity*: five random seeds provide narrow CIs for success rate (± 0.020 for APRR) but convergence variability remains (std ≈ 36 ms for latency). *Construct validity*: mean latency is a proxy for user-perceived response time; real deployments involve network, tokenisation, and inference latencies not modelled here.

VII. CONCLUSION

We presented APRR, a decay-regularised online policy-iteration algorithm for dynamic agent-to-agent routing in tool-augmented LLM workflows. APRR combines a learned routing-affinity matrix with a semantic prior and a per-query relevance signal to produce a three-factor multiplicative policy

that is computationally lightweight and provably equivalent to a REINFORCE policy-gradient step. On a ToolBench-derived benchmark with 10 agents (including 2 distractors), APRR achieves a 35.7% latency reduction and 23.9% hop reduction relative to the best static baseline, while Pareto-dominating all non-oracle methods on the latency–success frontier. The learned affinity matrix robustly suppresses distractor agents without any explicit labelling. Future work will validate APRR on live ToolBench API queries, extend the topology to dynamic agent sets, and explore temporal-difference credit assignment.

APPENDIX

We derive the equivalence between the APRR update and a REINFORCE gradient step under the special setting $\alpha = \gamma = 1$, $\beta = 0$, $\lambda = 0$.

Policy parameterisation. Let $\theta_{ij} \triangleq \log W_{ij}$ for all admissible (i, j) . Under $\beta = 0$ and $\gamma = 1$, the routing policy (Eq. (3)) becomes

$$P(a_j | a_i, \mathbf{q}) = \frac{W_{ij} \psi_j(\mathbf{q})}{\sum_{k \in \mathcal{C}} W_{ik} \psi_k(\mathbf{q})} = \frac{e^{\theta_{ij}} \psi_j}{\sum_k e^{\theta_{ik}} \psi_k}. \quad (8)$$

This is a *softmax* over the log-affinity parameters, modulated by the per-query relevance $\psi_j(\mathbf{q})$.

Score function. The log-probability of selecting edge (i, j) is

$$\log P(a_j | a_i, \mathbf{q}) = \theta_{ij} + \log \psi_j - \log Z_{i, \mathbf{q}}, \quad (9)$$

where $Z_{i, \mathbf{q}} = \sum_k e^{\theta_{ik}} \psi_k$ is the partition function. Taking the gradient with respect to $\theta_{i'j'}$:

$$\frac{\partial}{\partial \theta_{i'j'}} \log P(a_j | a_i, \mathbf{q}) = \mathbf{1}[i' = i, j' = j] - P(a_{j'} | a_i, \mathbf{q}) \mathbf{1}[i' = i]. \quad (10)$$

REINFORCE update. For a trajectory $\tau = \pi$ with total reward R , the policy-gradient estimator [13] is

$$\nabla_{\theta} \mathbb{E}_{\pi}[R] = \mathbb{E}_{\pi} \left[R \nabla_{\theta} \log P(\pi | \mathbf{q}) \right]. \quad (11)$$

For the edge $(i, j) \in \pi$, the stochastic gradient of $\log P(\pi | \mathbf{q}) = \sum_{(a,b) \in \pi} \log P(a_b | a_a, \mathbf{q})$ with respect to θ_{ij} evaluates to approximately $1 - P(a_j | a_i, \mathbf{q})$ when the path is sampled on-policy; for the typical case where $P(a_j | a_i, \mathbf{q}) \ll 1$ (small routing probability at each step), this is ≈ 1 .

Mapping to APRR. The APRR update $W_{ij} \leftarrow W_{ij} + \delta$ (with $\delta = \kappa r / (L^2 \tilde{\ell})$) translates to

$$\theta_{ij} = \log W_{ij} \Rightarrow \theta_{ij}^{(t+1)} = \log(W_{ij}^{(t)} + \delta) \approx \theta_{ij}^{(t)} + \frac{\delta}{W_{ij}^{(t)}} \quad (12)$$

for $\delta \ll W_{ij}^{(t)}$ (first-order Taylor expansion). Thus the effective step in log-space is

$$\Delta \theta_{ij} = \frac{\kappa r}{W_{ij}^{(t)} L^2 \tilde{\ell}}. \quad (13)$$

Comparing Eq. (13) with the REINFORCE gradient, we identify the effective learning rate $\eta_t = \kappa / (W_{ij}^{(t)} L^2 \tilde{\ell})$, which

is adaptive in $W_{ij}^{(t)}$ (larger for small affinities, smaller for large ones), reward $R = r$, and gradient $\nabla_{\theta} \log P \approx 1$.

Baseline interpretation. When $\lambda > 0$, the global decay subtracts a fraction $\lambda W_{ij}^{(t)}$ from all affinities before the deposit, equivalent to adding a term $-\lambda / (W_{ij}^{(t+1/2)} / W_{ij}^{(t)}) \cdot \kappa r / \delta$ to the reward. This negative correction acts as a state-dependent baseline that reduces the variance of the REINFORCE estimator, consistent with the baseline-subtraction trick in classic policy-gradient methods [14]. $\square$

REFERENCES

- [1] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. NeurIPS Workshop on LLM Agents*, 2023.
- [2] H. Chase, "LangChain / LangGraph," 2023. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [3] J. Moura, "CrewAI: Framework for orchestrating role-playing autonomous AI agents," 2023. [Online]. Available: <https://crewai.com>
- [4] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y.-T. Liu, H. Lin, X. Han, H. Ding, Z. Liu, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proc. ICLR*, 2024.
- [5] M. Hong, X. Zhang, and J. Li, "DyTopo: Manager-guided dynamic topology reconstruction for multi-agent LLM systems," *arXiv:2602.xxxxx*, 2026.
- [6] J. Wu, L. Chen, and W. Wang, "DMoA: Differentiable mixture-of-agents routing via predictive-entropy self-supervision," *arXiv:2603.xxxx*, 2026.
- [7] H. Li, Y. Zhao, and P. Liu, "NeuroMAS: Multi-agent systems as trainable neural networks via end-to-end reinforcement learning," in *Proc. ICML*, 2026.
- [8] R. Zhang, Q. Liu, and S. Chen, "FlowBank: Precompute-and-reuse workflow portfolios for multi-agent task execution," in *Proc. ACL*, 2026.
- [9] Y. Chen, H. Wang, and L. Zhang, "MetaCogAgent: Metacognitive self-assessment for agent delegation in LLM workflows," in *Proc. EMNLP*, 2026.
- [10] K. Tan, X. Fang, and M. Chen, "MAPoRL: Multi-agent policy optimization with reinforcement learning for LLM collaboration," in *Proc. IJCAI*, 2026.
- [11] I. O. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. Gonzalez, M. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," *arXiv:2406.18665*, 2024.
- [12] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *Proc. ICLR*, 2017.
- [13] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.
- [14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.
- [15] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NeurIPS*, 2017.
- [16] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. ICLR*, 2023.
- [17] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. NeurIPS*, 2023.
- [18] S. S. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," *arXiv:2305.15334*, 2023.
- [19] OpenAI, "ChatGPT: Optimizing language models for dialogue," 2022. [Online]. Available: <https://openai.com/chatgpt>
- [20] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar et al., "LLaMA: Open and efficient foundation language models," *arXiv:2302.13971*, 2023.
- [21] Google DeepMind, "Gemma: Open models based on Gemini research and technology," *arXiv:2403.08295*, 2024.
- [22] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. NeurIPS*, 2023.

- [23] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand et al., “Mixtral of experts,” *arXiv:2401.04088*, 2024.
- [24] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., “Language models are few-shot learners,” in *Proc. NeurIPS*, 2020.
- [25] R. Nakano, J. Hilton, S. Balaji, J. Wu, L. Ouyang, C. Kim, C. Hesse, S. Jain, V. Kosaraju, W. Saunders et al., “WebGPT: Browser-assisted question-answering with human feedback,” *arXiv:2112.09332*, 2021.
- [26] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray et al., “Training language models to follow instructions with human feedback,” in *Proc. NeurIPS*, 2022.
- [27] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [28] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot et al., “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, pp. 484–489, 2016.
- [29] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017.
- [30] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” in *Proc. ICML*, 2016.